

2. Briefly explain Symbol table?
uses of Symbol table? Operations
on symbol table? and methods
to implement symbol table?

Definition:

- o Defined as the set of Name and value pairs.
- o Important data structure created and maintained by the compiler in order to keep track of semantics of variables.
- o It stores information about the scope and binding information about names, information about instances of various entities such as variable and function names, classes, objects etc.
- o It is built-in lexical and syntax analysis phases.
- o The information is collected by the analysis phases of the

Date: _____

compiler and is used by the synthesis phases of the compiler to generate code.

- It is used by the compiler to achieve compile time efficiency.
- It is used by various phases of compiler as follows:

Lexical analysis:

- Creates new table entries in the table, for example like entries about tokens.

Syntax Analysis:

- Adds information regarding attribute type, scope, dimension, line of reference, use etc in the table.

Semantic analysis:

- Uses available information in the table to check for semantics.

Date: _____

ICR:

- Refers symbol table for knowing how much and what type of run-time is allocated and table helps in adding the temporary variable information.

Code Optimization:

- Uses information present in the symbol table for machine optimization.

Target code generation:

- Generates code by using address information of identifiers present in the table.

Symbol Table entries:

- Each entry in the symbol table is associated with attributes that support the compiler in different phases.

Item stored in symbol table:

- o Variable names and constants
- o Procedure and function names
- o Literal constants and strings
- o Compiler generated Temporaries
- o Labels in source language

Information used by the compiler from symbol table:

- o Data type and name
- o Declaring procedures
- o Offset in storage
- o if structure or record declared then, a pointer to structure table
- o For parameters, whether parameters passing by value or passing by reference
- o Number and type of arguments passed to function
- o Base address

Operations of symbol table:

→ The basic operations defined on symbol tables:

- o allocate
- o free
- o lookup
- o insert
- o set attribute
- o get attribute
- o Deletion

Uses of symbol table:

On Previous pages.

Date: _____

Day: MTWTF

Implementation of Symbol table:

⇒ There are four common way to implement symbol table:

- o Linked List

- o Hash Table

- o Linear Array

- o Binary Tree

(1) Linear List:

- o Simplest way to implement symbol table.

- o An array is used to store the information.

- o A pointer is maintained at end of all stored records and new names are added in the order as they arrive.

Pros:

- o Take less space.

- o Insertion is fast.

Date: _____

Day: MTWTF

Cons:

- o Look up is slow for large table.

	Name	Attribute	Searching
	id1	info1	
	id2	info2	
	...	...	
	idn	info n	↓
"Available" id →			← insert new name

(2) Linked List:

- o Also called self-organizing list.

- o A link field added to each record.

- o Searching of names is done in order pointed by the link of the link field.

- o A pointer "First" is maintained to point to the first record of the symbol table.

Date: _____ Day: (M T W T F S S)

Name	Attribute	link
id ₁	info ₁	
id ₂	info ₂	
id ₃	info ₃	
id ₄	info ₄	

First →

Available →

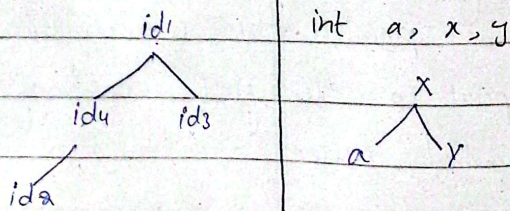
Searchings

- o fast

(3) Binary Tree:

- o Another approach is binary search tree.
- o Two link fields are used
 - left child
 - right child

Example:



searching:

fast

Hash table:

- o Most popular method to implement symbol table.
- o Two tables are maintained.
 - Hash table
 - Symbol table
- o A hash table is an array with an index range: 0 to table size - 1.
- o These entries are pointers pointing to the names of the symbol table.
- o To search for a name a hash function is used that will result in an integer b/w 0 and table size - 1.

Pros:

- o Insertion and lookup is fast

Cons:

Complicated

Date: _____

Day: M T W T F S

hash table		Name	Attribute	link
0	sum	sum	int	
1	i			
2	j	i	int	
...	...	j	int	
	ava	ava	float	
K-1				

Avail

⇒ Hash funⁿ $h(\text{name})$ return an integer value $0 \dots 9$.

Example:

int count;

char x[] = "Nero Academy"

Name	Type	Size	Dimension	LOD	LOU	Action
Count	int	2	0	...	...	--
x	char	12	1	---	--	--

Date: _____

Day: M T W T F S

Types of Symbol Table

⇒ There are two types of symbol table:

- o Global symbol table
- o Local^{scope} symbol table

⇒ Global symbol table can be accessed by all the procedures and scope symbol table.

⇒ Scope symbol table contains scope of variables.

Example:

int var;

int PROCA()

{

int var2, var3;

.....

}

int var4, var5;

.....

}

Date: _____

Day: M T W T F S

```
int var6;
```

```
----
```

```
{
```

```
----
```

```
int var7, var8;
```

```
----
```

```
}
```

```
}
```

```
int ProcB()
```

```
{
```

```
int var9, var10;
```

```
----
```

```
{
```

```
int var11, var12;
```

```
----
```

```
}
```

```
int var13;
```

```
}
```

Symbol Table (Global)

var1	var	int
ProcA	Proc	int
ProcB	Proc	int

Symbol Table (Proc A)

var2	var	int
var3	var	int
var6	var	int

Symbol Table (Proc B)

var9	var	int
var10	var	int
var13	var	int

var4	var	int
var5	var	int

Inner Scope-1

Symbol table

var7	var	int
var8	var	int

Inner scope-2

(Symbol table)

var11	var	int
var12	var	int

Inner-Scope-3

Symbol table

Date: _____

Day: Mon

Q. Create symbol table?

int x;

void f(int m)
{

float x, y;
}

int i, j;

int u, v;

}

}

int g(int n)
{

bool t;

}

Global
symbol table

x	var	int
f	fun	void
g	fun	int

func f

symbol
table

m	Par	int
x	var	float
y	var	float

func g symbol table

n	Par	int
t	var	bool

i	var	int
j	var	int

u	var	int
v	var	int

Run Time Environment

Q. Explain various source language issues?

⇒ These are various language features that affect the organization of memory.

⇒ The source language issues are:

- o Does the source language allow recursion?

- o How the parameters are passed to the procedure?

- o Does the procedure refer nonlocal names? How?

- o Does the memory support the memory allocation and deallocation dynamically?
